



Git – Intermediate

Module 8

Push/Pull

Collaboration

Git allows easy sharing of source code

On-premises

- GitHub Enterprise
- BitBucket

Online

- GitHub
- GitLab
- VSTS
- Others...

Push/pull

- Push
 - Push local changes to a central repository
- Pull
 - Pull changes from remote down to local repository
- Git does not allow pushing changes if the local branch is behind the remote branch. In such cases, Git will require a pull to fetch and merge changes from the remote before allowing the push.

Git pull

- git pull - Fetch from and integrate with another repository or a local branch
- Copies everything from remote repository to local computer

```
PS c:\> git pull [<options>] [<repository> [<refspec>...]]
```

Pull process



Incorporates changes from a remote repository into the current branch



`git pull` is shorthand for `git fetch` followed by `git merge FETCH_HEAD`



More precisely, `git pull` runs `git fetch` with the given parameters and calls `git merge` to merge the retrieved branch heads into the current branch



With `--rebase`, it runs `git rebase` instead of `git merge`

Git pull examples

- Update the remote-tracking branches for the repository you cloned from, then merge one of them into your current branch:

```
PS c:\> git pull  
PS c:\> git pull origin
```

- Merge into the current branch the remote branch next:

```
PS c:\> git pull origin next
```

Git pull examples cont.

- Leaves a copy of next temporarily in FETCH_HEAD but does not update any remote-tracking branches
- Using remote-tracking branches, the same can be done by invoking fetch and merge:

```
PS c:\> git fetch origin  
PS c:\> git merge origin/next
```


Git commit

- git commit - record changes to the repository
- Stores the current contents of the index in a new commit with a log message describing the

```
git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>] [--amend]
           [--dry-run] [(-c | -C | --fixup | --squash) <commit>]
           [-F <file> | -m <msg>] [--reset-author] [--allow-empty]
           [--allow-empty-message] [--no-verify] [-e] [--author=<author>]
           [--date=<date>] [--cleanup=<mode>] [--[no-]status]
           [-i | -o] [-S[<keyid>]] [--] [<file>...]
```

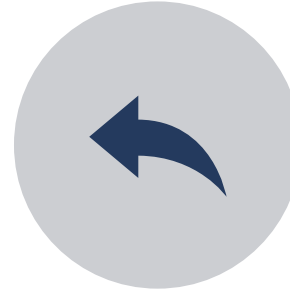
Commits best practices

- Everyone commits to the mainline every day
- Integration and communication are key
- Changes need to be communicated
- Builds pass tests
- Committed frequently to identify potential conflicts rapidly
- Frequent commits encourage smaller chunks to be built/updated
- Without frequent checks, conflicts or bugs can fester for weeks and cause bigger issues later in the process
- Fix broken builds immediately

Commits by design



Encourages you to be cohesive



Decoupled from other previous commits



Good idea to begin the commit message with a single short (less than 50 character) description, followed by a blank line and then a more thorough description



Text up to the first blank line, in a commit message, is treated as the commit title, and that title is used throughout Git

Git push

- git push - Update remote refs along with associated objects
- Updates remote refs using local refs, while sending objects necessary to complete the given refs

```
git push [--all | --mirror | --tags] [--follow-tags] [--atomic] [-n | --dry-run] [--  
receive-pack=<git-receive-pack>]  
        [--repo=<repository>] [-f | --force] [-d | --delete] [--prune] [-v | --verbose]  
        [-u | --set-upstream] [-o <string> | --push-option=<string>]  
        [--[no-]signed|--signed=(true|false|if-asked)]  
        [--force-with-lease[=<refname>[:<expect>]]]  
        [--no-verify] [<repository> [<refspec>...]]
```

Push process



GIT PUSH



UPLOADS TO REMOTE REPOSITORY
FROM LOCAL COMPUTER

Git push examples

- `git push <remote>`, where `<remote>` is the current branch's remote (or origin, if no remote is configured for the current branch)

```
PS c:\> git push  
PS c:\> git push <remote>
```

- Find a **ref** that matches master in the source repository (most likely it finds **refs/heads/master**), and update the same **ref** in origin repository
- If master did not exist remotely, it would be created

```
PS c:\> git push origin master
```

Git reset

- Be a good steward with your code
- `git reset --hard HEAD` allows reverted changes
- Makes changes to the working tree
- `git rev-parse`
 - Parsing engine that runs commits
 - Also runs other under the hood process

Git reset details

- First and second form, copy entries from <tree-ish> to the index
- Third form, set the current branch head (HEAD) to <commit>
- Optionally modifying index and working tree to match
- <tree-ish>/<commit> defaults to HEAD in all forms

```
git reset [-q] [<tree-ish>] [--] <paths>...  
git reset (--patch | -p) [<tree-ish>] [--] [<paths>...]  
git reset [--soft | --mixed [-N] | --hard | --merge | --keep] [-q] [<commit>]
```


Git reflog

- git reflog (“get out of jail free card”)
 - Provides GUID values of recent activity
 - It is a history of activity
- Be clear on which branch you are on when re-running/backing out the history

```
git reflog <subcommand> <options>
```

```
git reflog [show] [log-options] [<ref>]
```

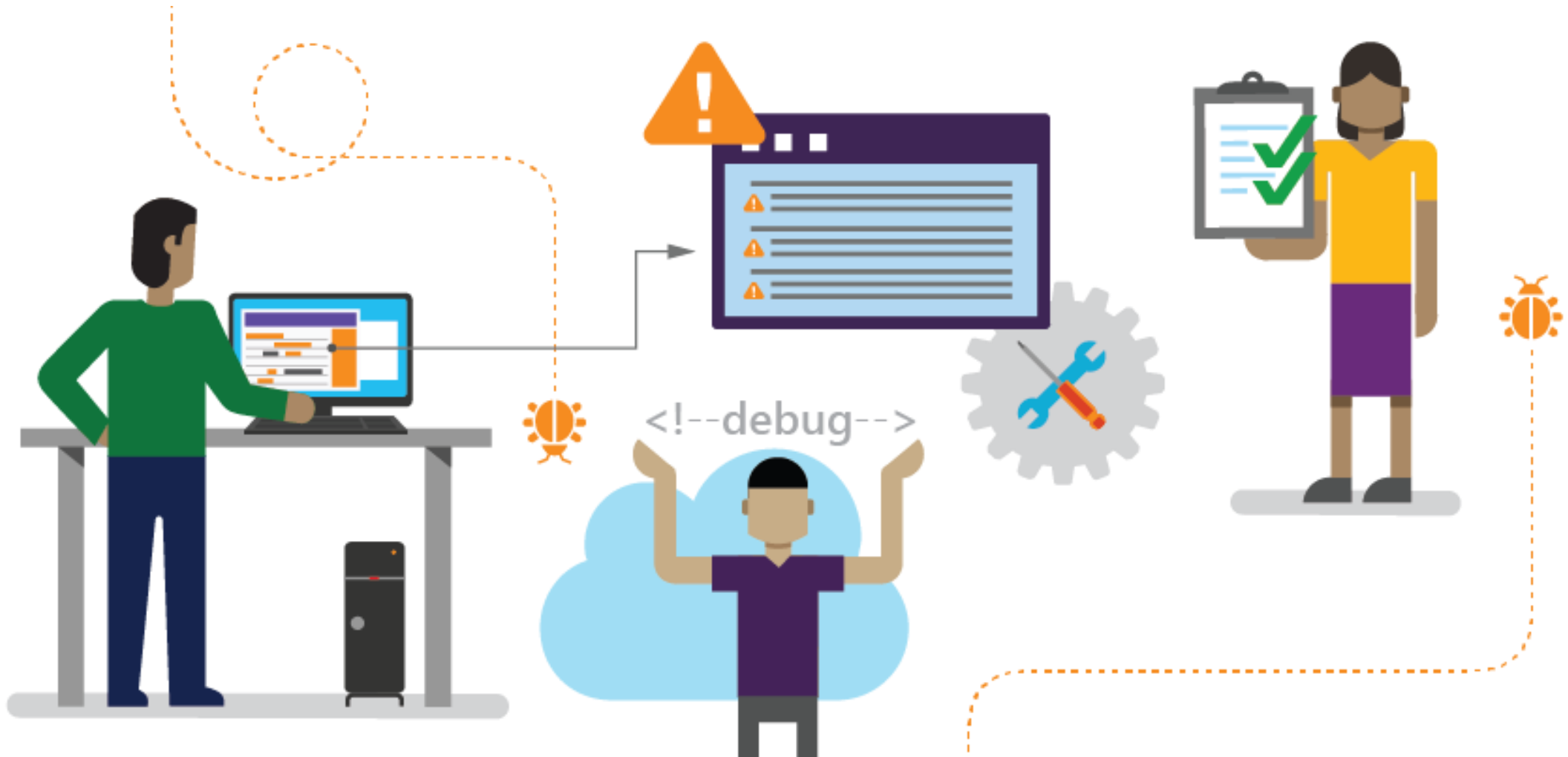
```
git reflog expire [--expire=<time>] [--expire-unreachable=<time>]  
    [--rewrite] [--updateref] [--stale-fix]  
    [--dry-run | -n] [--verbose] [--all | <refs>...]
```

```
git reflog delete [--rewrite] [--updateref]  
    [--dry-run | -n] [--verbose] ref@{specifier}...
```

```
git reflog exists <ref>
```

Branch and merging options

Development Struggles



Typical Lifecycle

Source
Repository

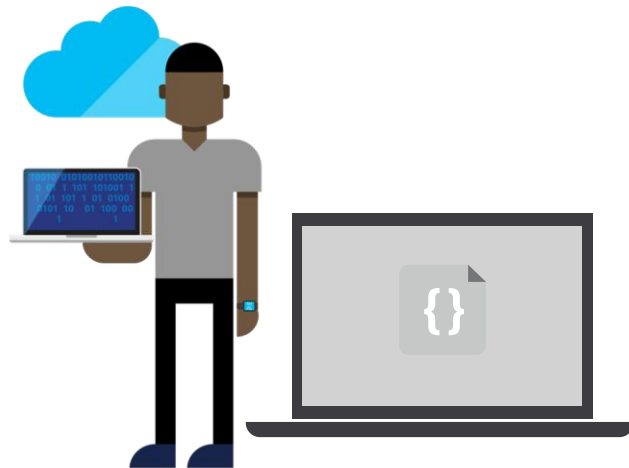
Build



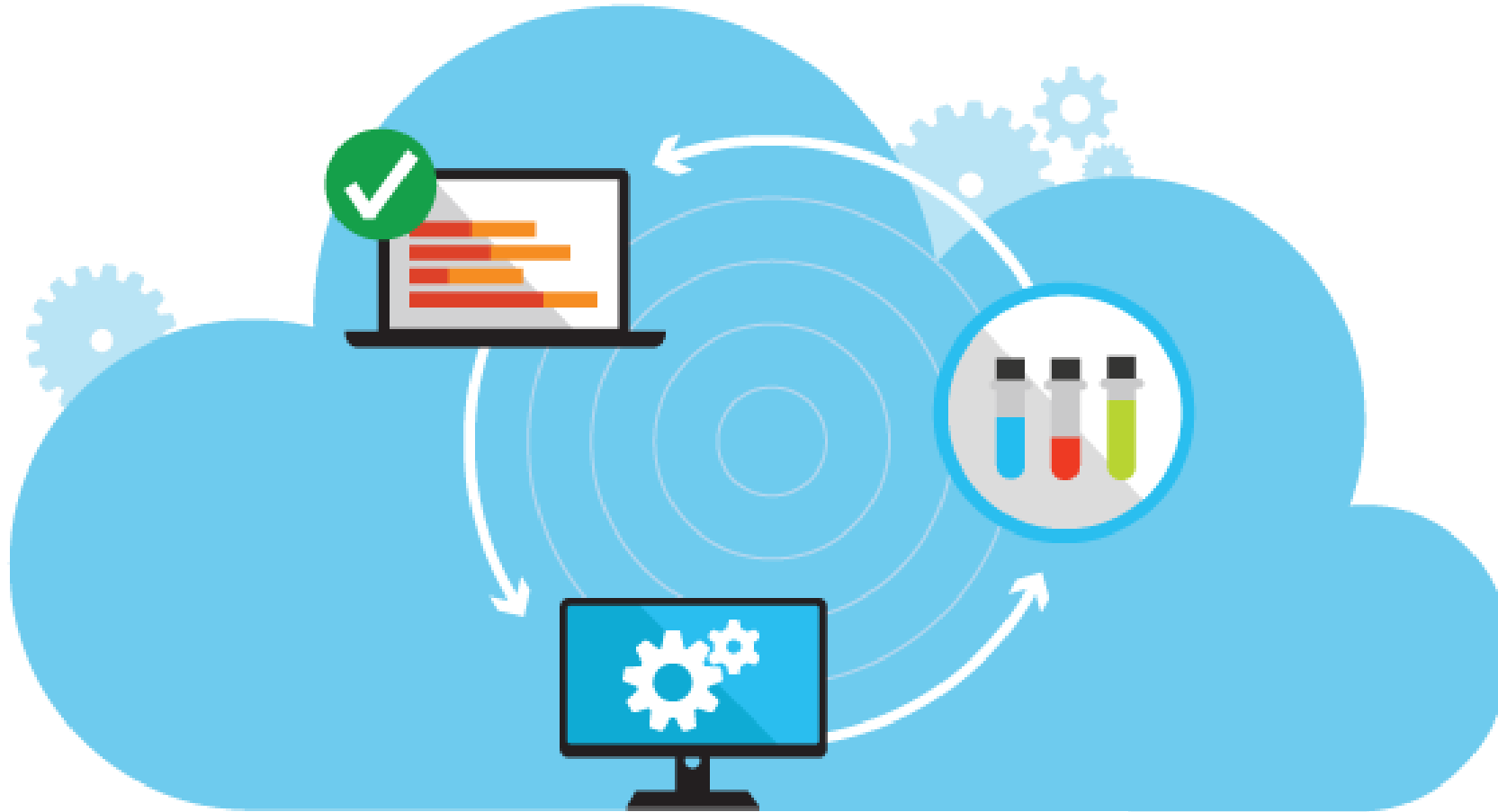
Production



The QA
Environment



Continuous Integration



Importance of Continuous Integration

- Continuous Integration should be the backbone of development process
- Continuous Integration results in:
 - Improved quality
 - Improved productivity
 - Reduced risk

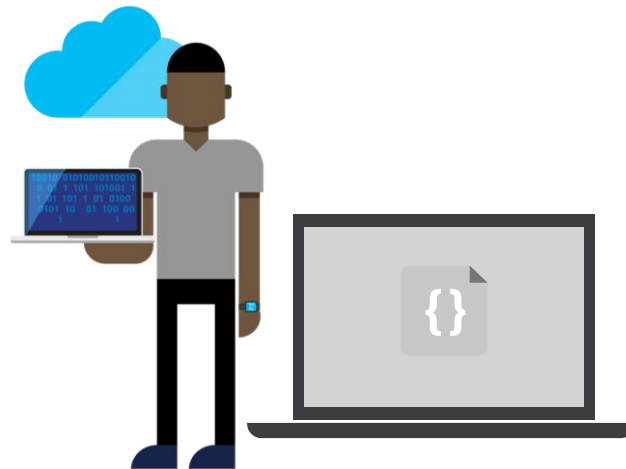
Continuous Integration – How It Works

Source
Repository

Build



Test



Branching high level



Work can be done in branches



Each branch is a separate line of development



Does not affect any other branch



Branches point to different commits

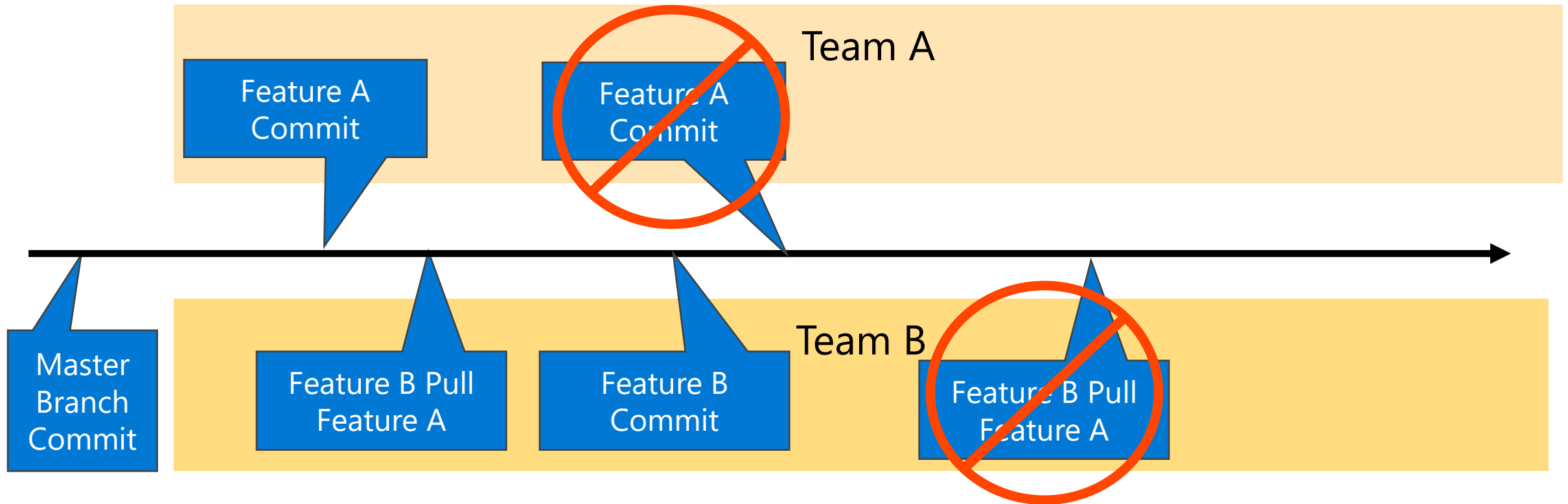


Initial branch is called master

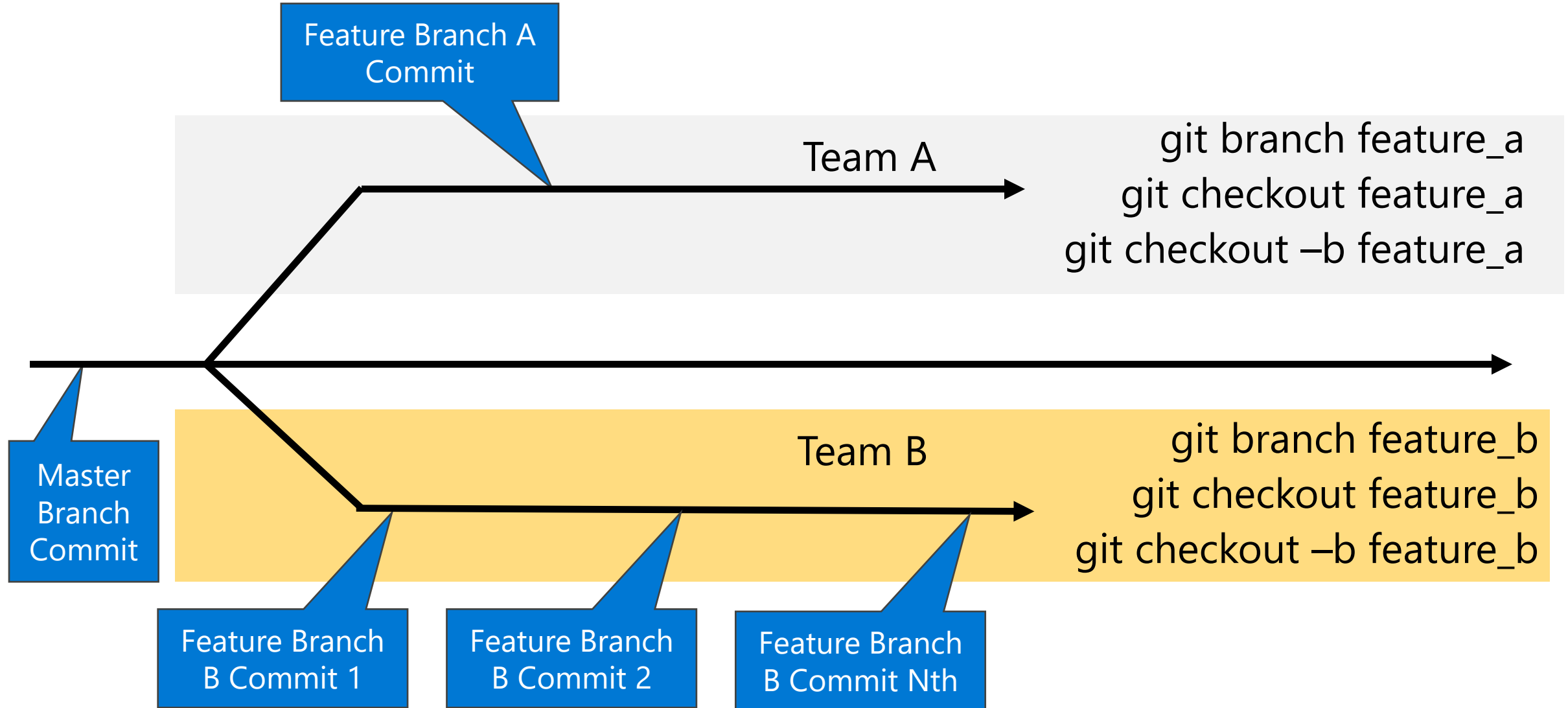
Branch – HEAD

- Branches stored under the refs/HEADS tree
 - records tip-of-the-tree commit objects of branch name
 - `cat .git/HEAD`
 - Blobs stored as hashes: `a113a6c7c3682458ec6307a7c2c350ed849bfaf7`

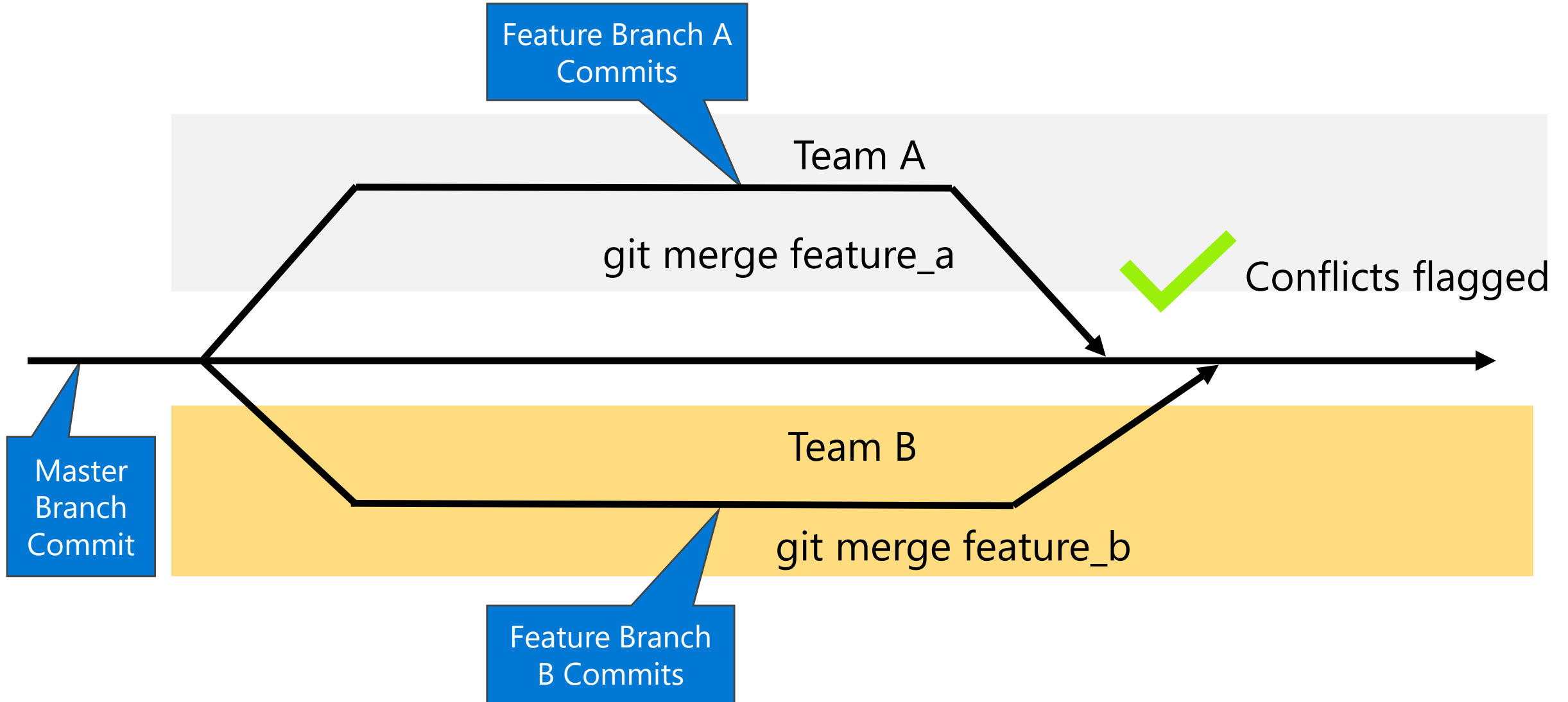
Branching – why needed



Branching – solution



Branching – merging cont.



New branch

- Add a new branch – `git branch <<branch name>>`
- `git checkout <<branch name>>`

```
PS c:\MyNewRepository> git branch dev  
PS c:\MyNewRepository> git checkout dev  
Switched to a new branch 'dev'
```

- Create and switch to – `git checkout -b <<branchname>>`

```
PS c:\MyNewRepository> git checkout -b dev  
Switched to a new branch 'dev'
```

Viewing branches

- View all local branches – git branch

```
PS c:\MyNewRepository> git branch
* dev
master
```

- View local and remote branches

```
PS c:\MyNewRepository> git branch --all
* dev
master
remotes/origin/master
```

Git status

- Displays paths that have differences between the index file and the current HEAD commit
- Paths that have differences between the working tree and the index file
- Paths in the working tree that are not tracked by Git

```
git status [<options>...] [--] [<pathspec>...]
```

Branching – merging

- Brings together two branches
- Fast forward or recursive types
- Choose to keep or squash commits in topics branch
- Merge conflicts prevent changes being overwritten

Git merge

- `git merge <<Branch Name>>`

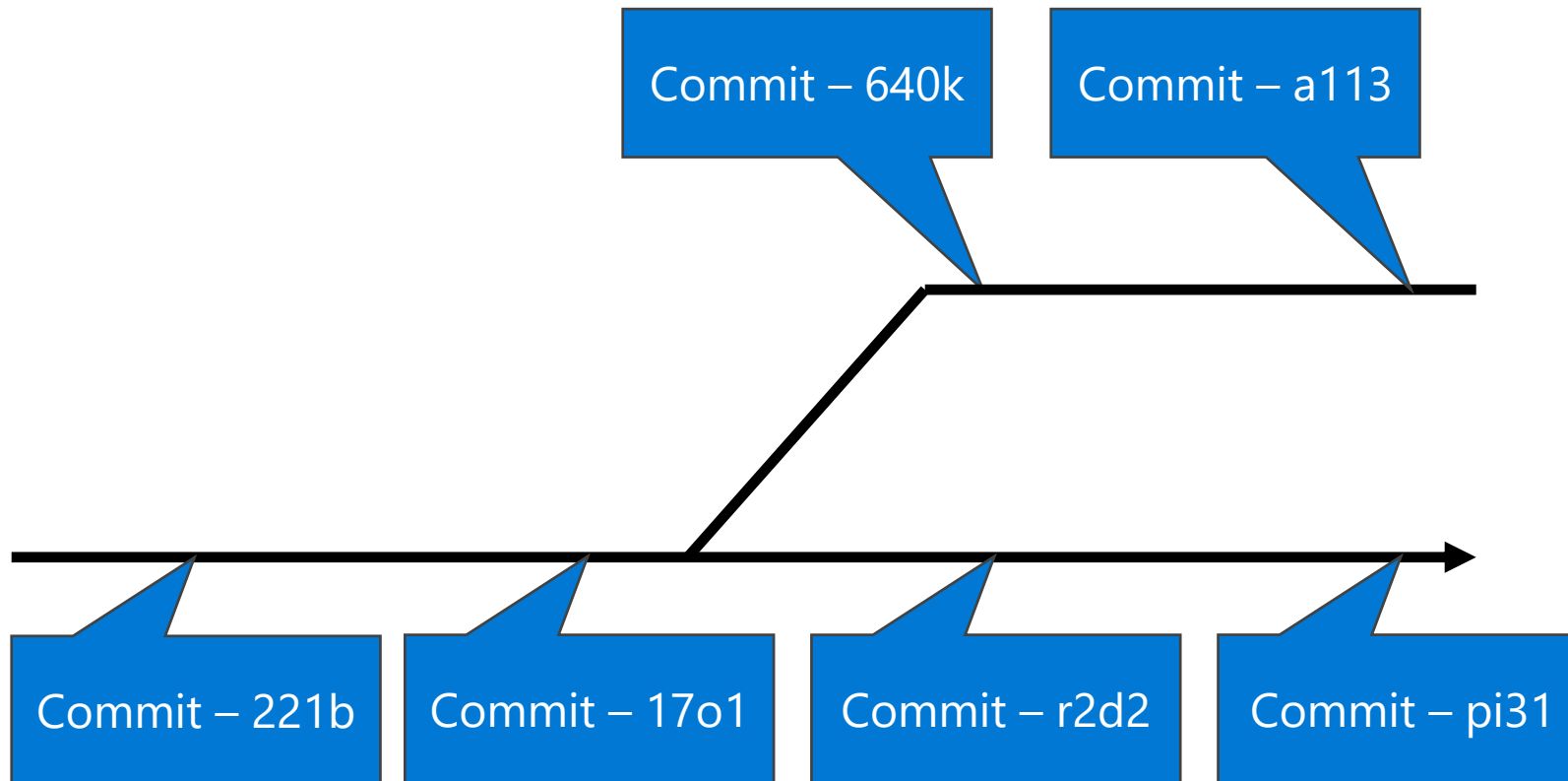
```
PS c:\MyNewRepository> git merge dev
Updating 5e9c4cb..b5cfbf5
Fast-forward
 file2.txt | 0
 1 file changed, 0 insertions(+), 0 deletions (-)
 create mode 100644 file2.txt
PS c:\MyNewRepository>
```

```
PS c:\MyNewRepository> git merge dev
Merge made by the 'recursive' strategy
 file2.txt | 1 +
 1 file changed, 1 insertions(+)
PS c:\MyNewRepository>
```

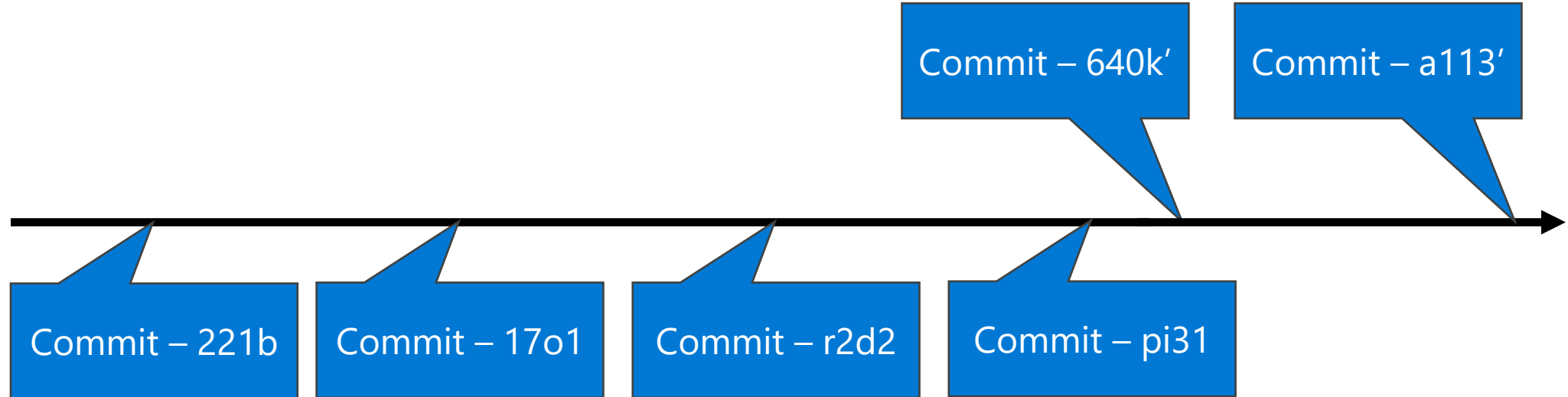
Git rebase

- Reapply all commit(s) from your branch to the tip of another branch
- References just a few characters of a hash
 - Ex: a113 to call: a113a6c7c3682458ec6307a7c2c350ed849bfaf7
- Reapplying commits, git creates new ones
- New commits, even if bringing the same set of change(s), will be treated as completely independent and different
- git rebase reapplies commits, while not destroying old ones
- Thus, after a rebase, old commits will still be in the /objects folder in .git directory
- Feature branch has completely new commits, with same changes, but completely different objects

Branching – rebase before



Branching – rebase after



Git – Intermediate

(30 minutes)

- Clone
- Push/Pull
- Branch
- Merging

